CS-1101
PROGRAMMING FUNDAMENTALS

Lecture 02

By

Engr. Muhammad Awais

Oct 21, 2025

Lecture Contents

- Recap of Lecture 1
- Problem Solving Techniques
- Algorithms
- Flowcharts
- Examples of Algorithm's and Flowcharts
- Quiz # 1

Lecture Objectives

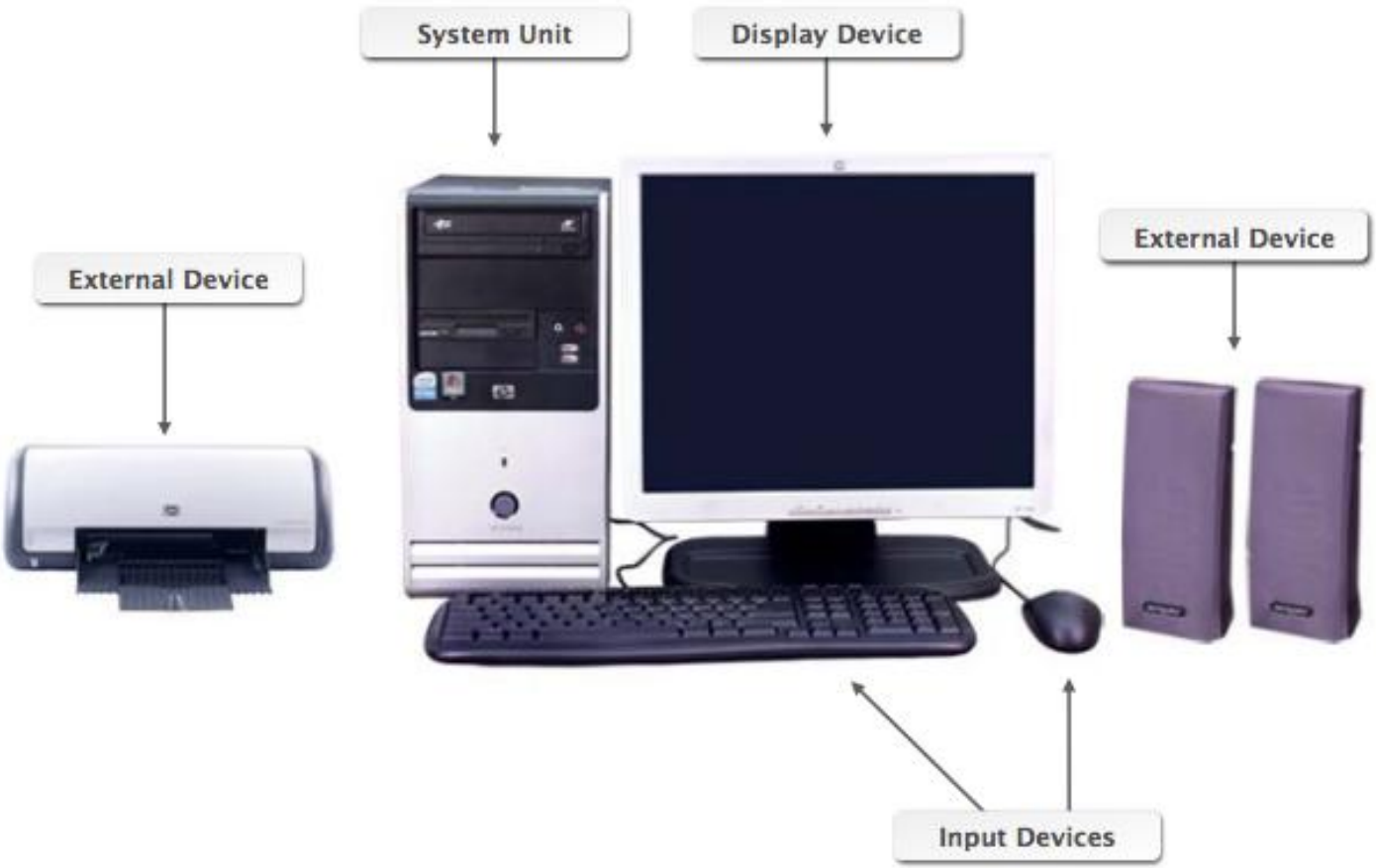
- Learn what an algorithm is and explore problem-solving techniques.
- Learn to draw a flowchart to illustrate an algorithm

RECAP LECTURE #1

WHAT IS A COMPUTER ?

- A computer is a programmable electronic device that accepts raw data as input and processes it with a set of instructions (a program) to produce the result as output.
- Two main components of computer are:
  - Hardware
  - Software

HARDWARE


System Unit
Display Device
External Device
External Device
Input Devices

SOFTWARE

Software
Mac OS
Linux
iOS
ANDROID
System Software
Ps
Adobe Acrobat
Office
Application Software

COMPUTER PROGRAM

- A computer program is a sequence of instructions written using a Computer Programming Language to perform a specified task by the computer.

The Evolution of Programming Languages

- Low-level Language
  - Machine Language
  - Assembly Language
- High-level languages include Basic, FORTRAN, COBOL, Pascal, C, C++, C#, and Java.

TRANSLATORS

- **Assembler:** translates a program written in a assembly language to machine language.
- **Compiler:** translates a program written in a high-level language to machine language.

Interpreter ?

Processing a C++ Program (cont'd.)

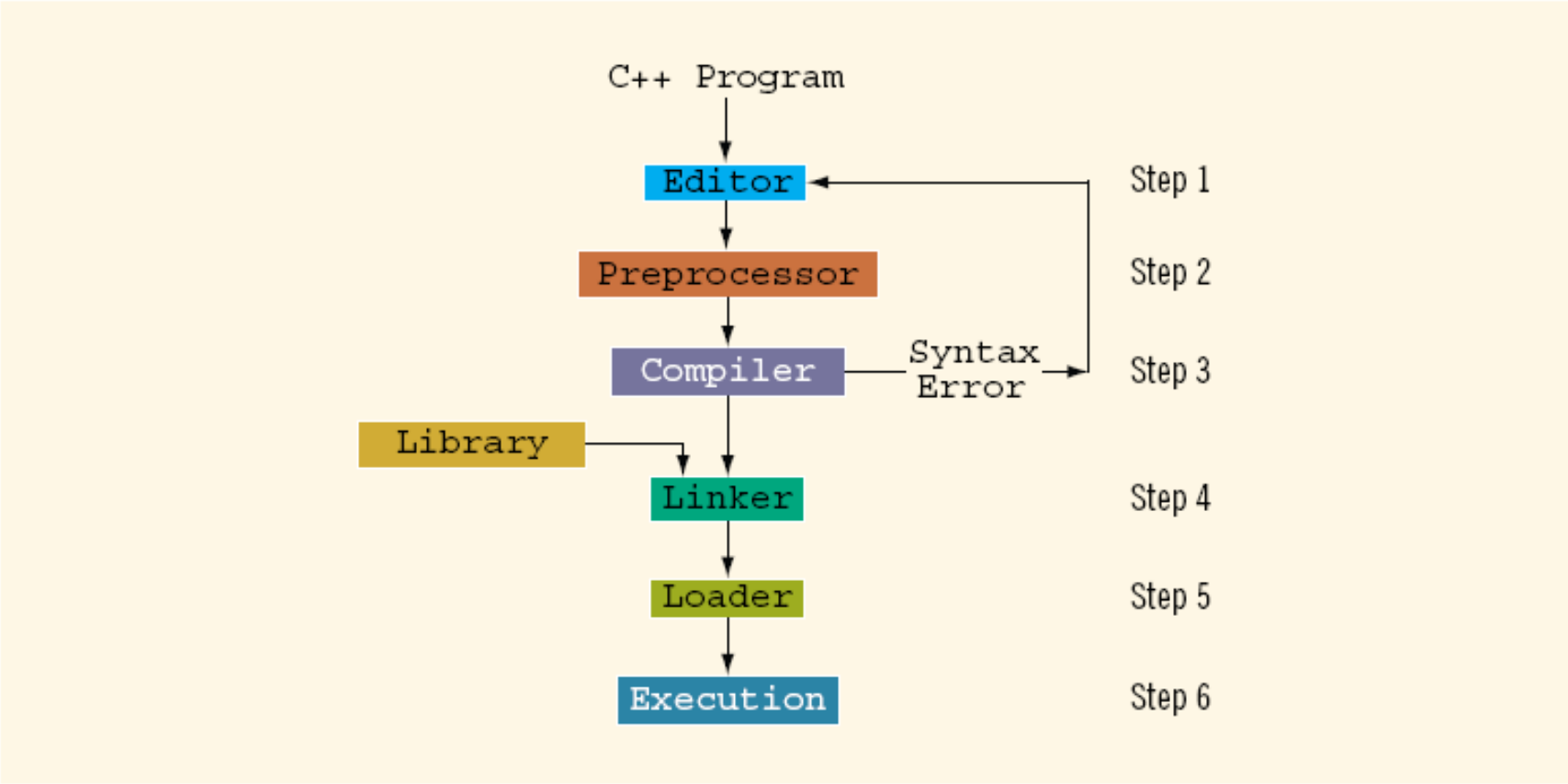


FIGURE 1-3 Processing a C++ program

LECTURE # 2

WHAT THIS PICTURE DEPICTS?

Faulty Vehicle – Problem Solving

- IDENTIFY the problem.
- ANALYZE the issue.
- PLAN the steps.
- IMPLEMENT the plan.
- TEST if okay then issue is resolved.

Programming with the Problem Analysis–Coding–Execution Cycle

- Programming is a process of problem solving
- One problem-solving technique:
  - Identify/Define the problem.
  - Analyze the problem
  - Design steps (algorithm) to solve the problem
- <u>Algorithm</u>:
  - Step-by-step problem-solving process
  - Solution achieved in finite amount of time

The Problem Analysis–Coding–Execution Cycle (cont'd.)

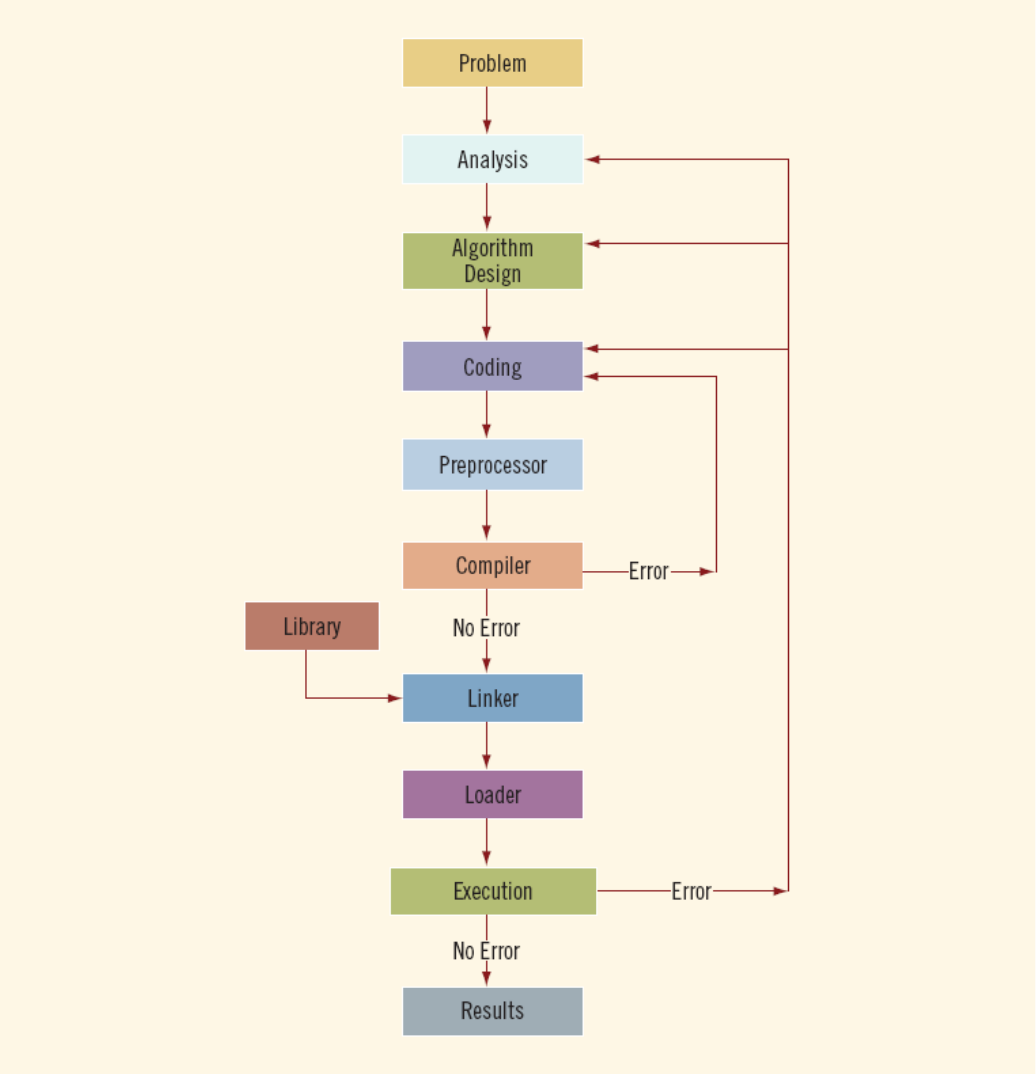


FIGURE 1-4 Problem analysis–coding–execution cycle

The Problem Analysis–Coding–Execution Cycle (cont'd.)

- Run code through compiler
- If compiler generates errors
  - Look at code and remove errors
  - Run code again through compiler
- If there are no syntax errors
  - Compiler generates equivalent machine code
- Linker links machine code with system resources

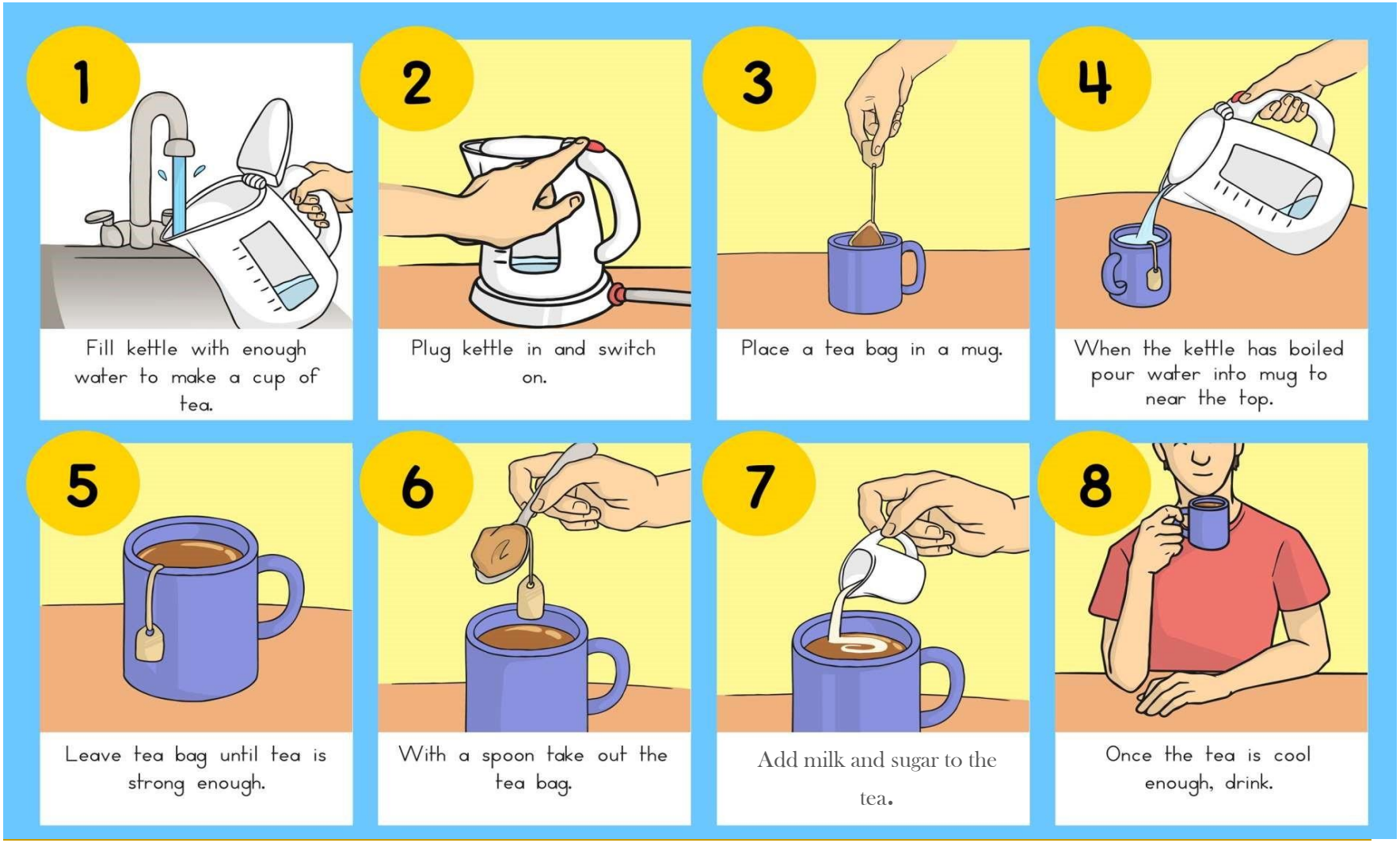
The Problem Analysis–Coding–Execution Cycle (cont'd.)

- Once compiled and linked, loader can place program into main memory for execution
- The final step is to execute the program
- Compiler guarantees that the program follows the rules of the language
  - Does not guarantee that the program will run correctly

Algorithm

CAN YOU WRITE STEPS TO MAKE A CUP OF TEA?

Algorithm


1
Fill kettle with enough water to make a cup of tea.
2
Plug kettle in and switch on.
3
Place a tea bag in a mug.
4
When the kettle has boiled pour water into mug to near the top.
5
Leave tea bag until tea is strong enough.
6
With a spoon take out the tea bag.
7
Add milk and sugar to the tea.
8
Once the tea is cool enough, drink.

Algorithm:

- **Step-1** Start
- **Step-2** Fill Kettle with enough water to make a cup of tea.
- **Step-3** Plug Kettle in and switch on.
- **Setp-4** Place a tea bag in a mug.
- **Setp-5** When the water has boiled pour water into mug near to top .
- **Setp-6** Leave a bag in a mug until tea is strong enough.
- **Setp-7** With a spoon takeout the teabag.

- **Step-8** Add milk and sugar to the tea.
- **Setp-9** Once the tea is cool enough , drink it.
- **Setp-10** Stop

Example 1

- Write an algorithm to display your name on the screen?
  - **Step-1** Start
  - **Step-2** Print “Awais”
  - **Step-3** Stop

Example 2

- Design an algorithm to add two numbers 10 and 20. Also display the result ?
  - **Step-1** Start
  - **Step-2** Initialize variable A =10
  - **Step-3** Initialize variable B =20
  - **Step-4** Add A & B and store their result into sum.

    `sum = A + B`
  - **Step-5** Print (sum)
  - **Step-6** Stop

Example 3

- Design an algorithm to multiply three numbers 10,5,2?
  - **Step-1** Start
  - **Step-2** Initialize variable A =10
  - **Step-3** Initialize variable B =5
  - **Step-4** Initialize variable C =2
  - **Step-5** Multiply A,B,C and store the result in D.

    `D = A x B x C`
  - **Step-6** Print (D)
  - **Step-7** Stop

Example 4

- Design an algorithm to add two numbers inputted by user?
  - **Step-1** Start
  - **Step-2** Input two numbers A,B
  - **Step-3** Add A & B and store their result into sum.

    ```
    sum = A + B
    ```

  - **Step-4** Print (sum)
  - **Step-5** Stop

Example 5

- Design an algorithm to find the smallest number from two numbers?
  - **Step-1** START
  - **Step-2** INPUT two numbers A,B
  - **Step-3** IF A < B PRINT(A) OTHERWISE PRINT (B)
  - **Step-4** STOP

Example 5 – More Detailed

- Design an algorithm to find the smallest number from two numbers?
  - **Step-1** START
  - **Step-2** INPUT two numbers A,B
  - **Step-3** IF A < B

    Print (A)

    ELSE

    Print (B)

    ENDIF
  - **Step-4** STOP

Example 6

- Design an algorithm to find the smallest number from three numbers?
  - **Step-1** START
  - **Step-2** INPUT three numbers A,B,C
  - **Step-3** IF A < B

```
IF A < C
        Print (A)
ELSE
        Print(C)
END IF
```

Example 6 cont'd.

```
ELSE
        IF B < C
                Print (B)
        ELSE
                Print (C)
        END IF
END IF
```

- **Step-4** Stop

Flowcharts

- A flowchart is a graphical representation of an algorithm.
- Flowcharts use simple geometric shapes to depict processes and arrows to show relationships and process/data flow.

Flowchart Symbols

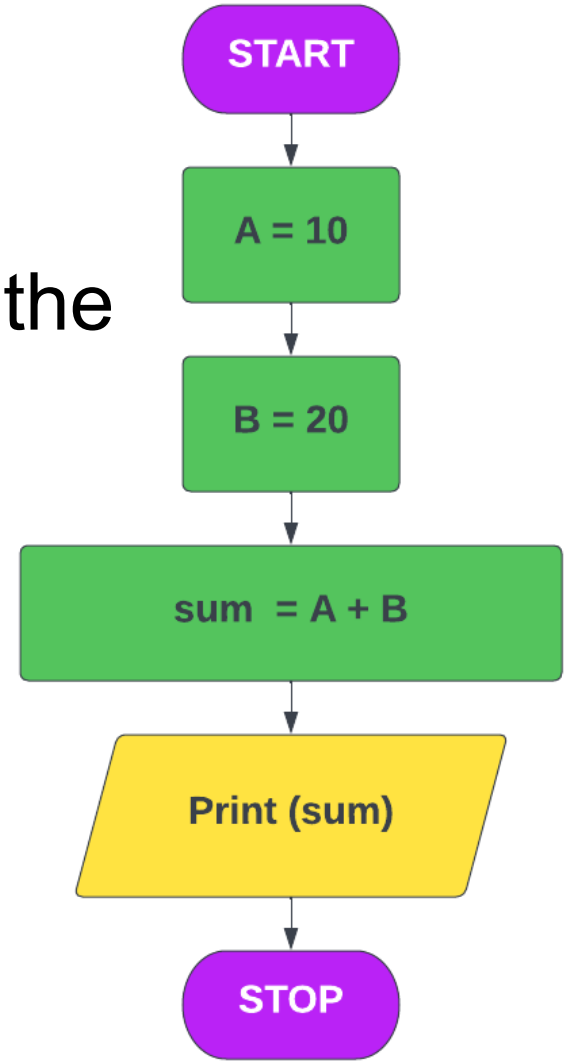
Symbol	Symbol Name	Purpose
	Start/Stop	Used at the beginning and end of the algorithm to show start and end of the program.
	Process	Indicates processes like mathematical operations.
	Input/ Output	Used for denoting program inputs and outputs.
	Decision	Stands for decision statements in a program, where answer is usually Yes or No.
	Arrow	Shows relationships between different shapes.
	On-page Connector	Connects two or more parts of a flowchart, which are on the same page.
	Off-page Connector	Connects two parts of a flowchart which are spread over different pages.

Guidelines for developing flowcharts

- Flowchart can have only one start and one stop symbol
- On-page connectors are referenced using letters
- Off-page connectors are referenced using numbers
- General flow of processes is top to bottom or left to right
- Arrows should not cross each other

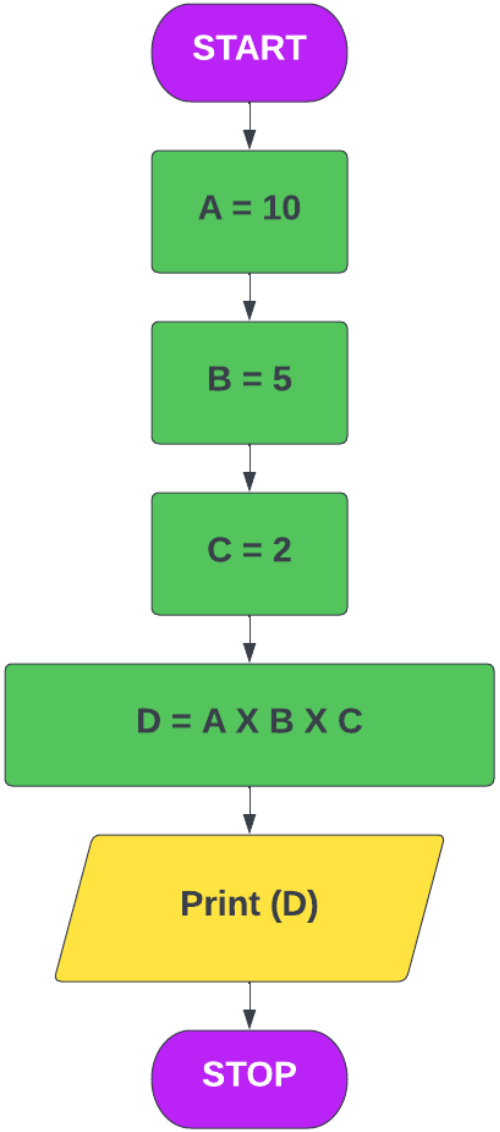
Example 1

- Draw a flowchart to add two numbers 10 and 20. Also display the result ?



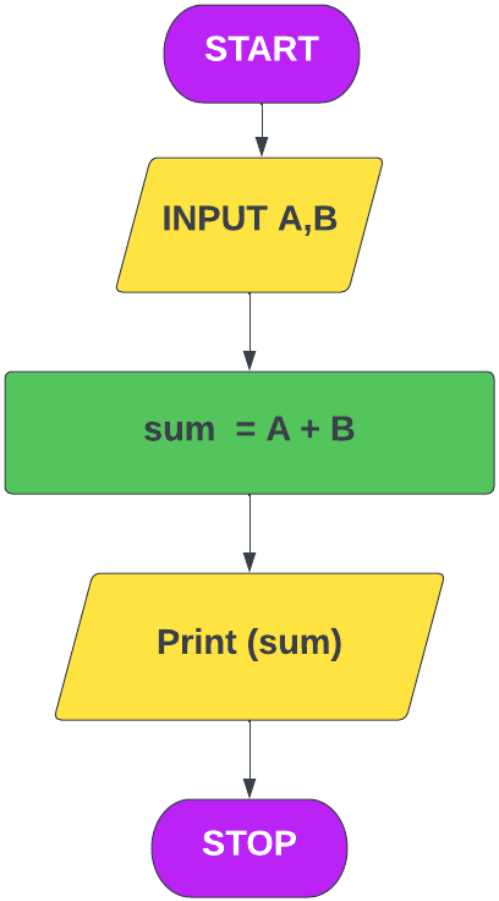
Example 2

- Draw a flowchart to multiply three numbers 10,5,2?



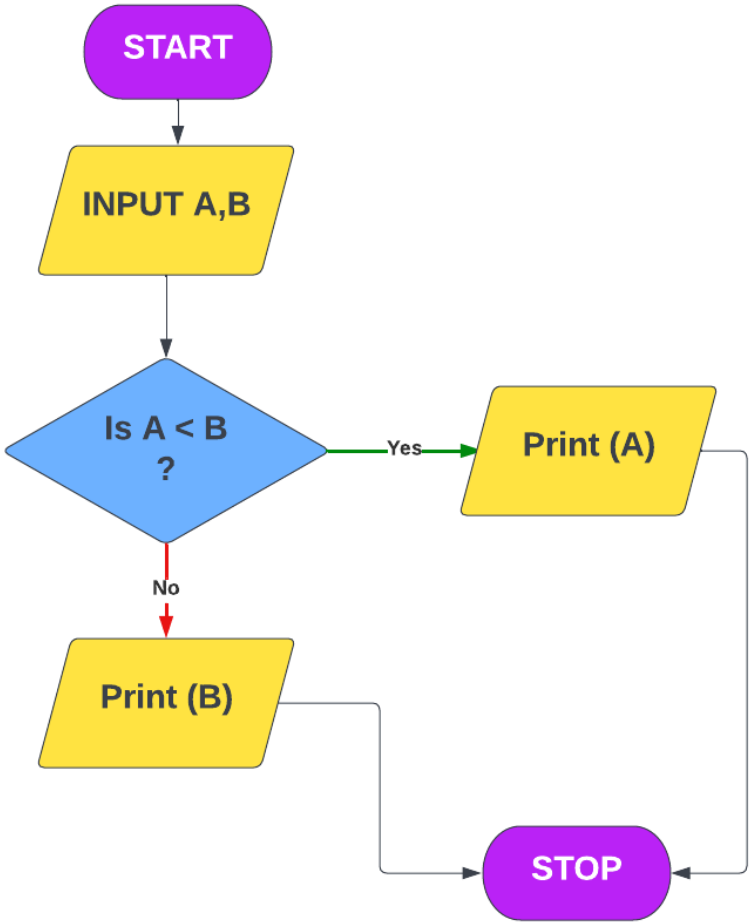
Example 3

- Draw a flowchart to add two numbers inputted by user?



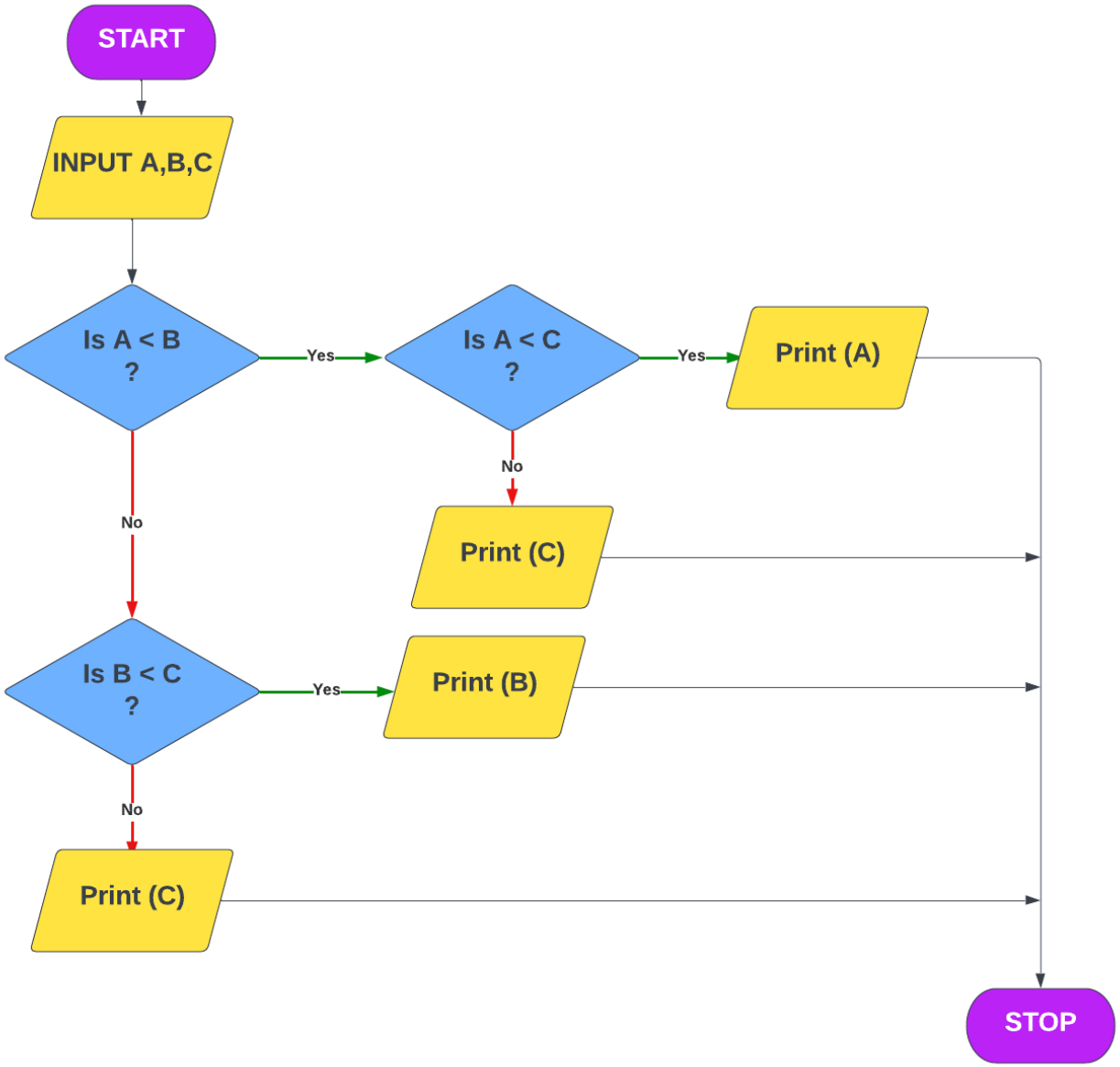
Example 5

- Draw a flowchart to find the smallest number from two numbers?



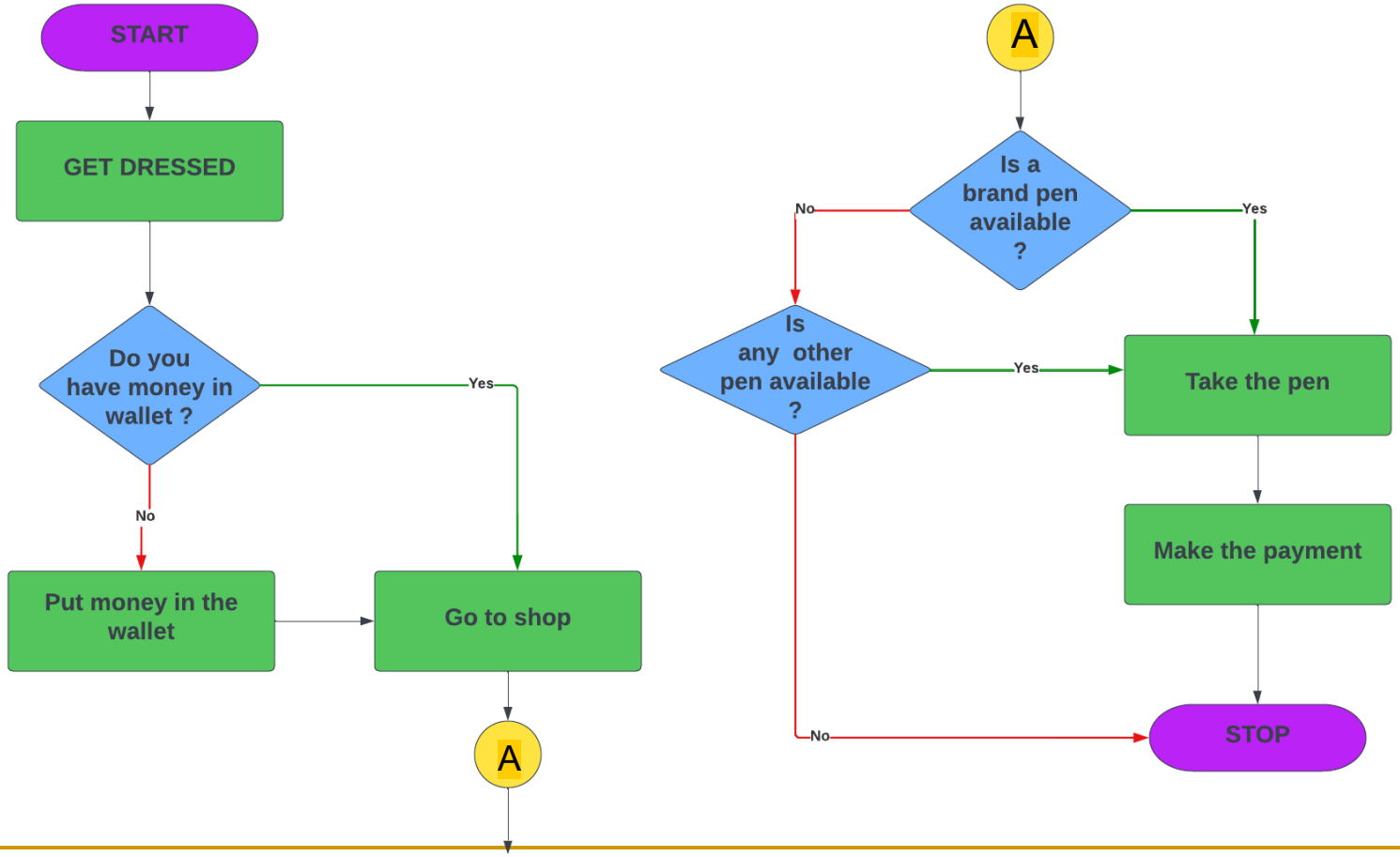
Example 6

- Draw a flowchart to find the smallest number from three numbers?


START
INPUT A,B,C
Is A < B ?
Yes
Is A < C ?
Yes
Print (A)
No
Print (C)
No
Is B < C ?
Yes
Print (B)
No
Print (C)
STOP

On Page Connector

- Example Flow chart to buy a pen from shop.



Off Page Connector

- Example Flow chart to buy a pen from shop.

REFERENCES

- https://www.tutorialspoint.com/programming_methodologies/programming_methodologies_flowchart_elements.htm
- Algorithm and Flowchart Tutorial with Examples
- https://lucid.app/